Криптографія

Комп’ютерний практикум №3

Криптоаналіз афінної біграмної підстановки

Фризюк Денис та Лапінський Вадим ФБ-41

Варіант 6

Мета: Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці

Хід роботи

1. Реалізувати підпрограми із необхідними математичними операціями: обчисленням оберненого елементу за модулем із використанням розширеного алгоритму Евкліда, розв'язуванням лінійних порівнянь. При розв'язуванні порівнянь потрібно коректно обробляти випадок із декількома розв'язками, повертаючи їх усі.

```
9   def ext_gcd(a, b):
10      if a == 0:
11          return b, 0, 1
12      gcd, x1, y1 = ext_gcd(b % a, a)
13      x = y1 - (b // a) * x1
14      y = x1
15      return gcd, x, y
```

Функція являє собою рекурсивну реалізацію розширеного алгоритму Евкліда, яка знаходить не лише найбільший спільний дільник (НСД) двох чисел, а й відповідні коефіцієнти Безу x та y, що задовольняють лінійне рівняння a * x + b * y = НСД. Алгоритм працює за таким принципом: спочатку перевіряється базова умова зупинки якщо параметр a дорівнює нулю, рекурсія завершується, і функція повертає значення b як НСД разом із базовими коефіцієнтами 0 та 1. Якщо ж a не дорівнює нулю, функція здійснює рекурсивний виклик самої себе з аргументами b % a та a, реалізуючи таким чином класичне ділення з остачею. Під час повернення з кожного рівня рекурсії поточні коефіцієнти перераховуються за формулами x = y_1 - (b // a) * x_1 та y = x_1. У контексті виконання даної лабораторної роботи, одержаний коефіцієнт x відіграє головну роль: після приведення його за робочим модулем, він стає шуканим оберненим елементом a^-1, який використовується для розв'язування лінійних порівнянь і дешифрування тексту.

```
17  def modinv(a, m):
18      g, x, y = ext_gcd(a, m)
19      if g != 1:
20          return None
21      return x % m
```

Для програмної реалізації безпосереднього знаходження оберненого елемента за модулем ми написали функцію modinv(a, m). Вона є логічним продовженням розширеного алгоритму Евкліда, математичне обґрунтування якого наведено у теоретичних відомостях до лабораторної роботи. На початку свого виконання функція викликає підпрограму ext_gcd(a, m), звідки отримує значення найбільшого спільного дільника g та відповідні лінійні коефіцієнти x і y. Згідно з теорією чисел, обернений елемент існує виключно за умови, що числа є взаємно простими. З огляду на це, в алгоритмі передбачено перевірку базової умови: якщо обчислений НСД g не дорівнює одиниці, виконання переривається і функція повертає значення None, відкидаючи невалідного кандидата на ключ. Якщо ж необхідна умова виконується, програма успішно завершує роботу і повертає коефіцієнт x, приведений за робочим модулем m за допомогою стандартної операції x % m, що і є остаточним значенням шуканого оберненого елемента.

$$u_0 = 1,\ u_1 = 0,\ u_{i+1} = u_{i-1} - q_i u_i;$$
$$v_0 = 0,\ v_1 = 1,\ v_{i+1} = v_{i-1} - q_i v_i.$$

Звідси обернений елемент до числа a за модулем n знаходиться таким чином: оскільки a обертається лише за умови $\gcd(a,n) = 1$, то за розширеним алгоритмом Евкліда знаходяться такі числа u та v, що $au + nv = 1$. Звідси $au \equiv 1 \pmod n$ та $u \equiv a^{-1} \pmod n$.

```
23  def solve_linear_congruence(a, b, n):
24      g, x, y = ext_gcd(a, n)
25      if b % g != 0:
26          return []
27      a1, b1, n1 = a // g, b // g, n // g
28      x0 = (b1 * modinv(a1, n1)) % n1
29      return [(x0 + i * n1) % n for i in range(g)]
```

Для розв'язання лінійних порівнянь виду $ax \equiv b \pmod n$ ми реалізували функцію solve_linear_congruence(a, b, n). Спочатку алгоритм визначає найбільший спільний дільник g між коефіцієнтом a та модулем n за допомогою підпрограми ext_gcd. Згідно з алгоритмом розв'язування порівнянь, якщо вільний член b не ділиться на знайдений НСД без остачі (умова b % g != 0), система не має розв'язків, тому програма одразу повертає порожній масив. У випадку наявності розв'язків, початкові параметри рівняння спрощуються шляхом цілочисельного ділення на g, утворюючи допоміжні змінні a1, b1 та n1. Далі обчислюється базовий корінь x0 через знаходження оберненого елемента modinv(a1, n1) та його множення на b1. Оскільки за такого значення НСД порівняння має рівно g розв'язків, на фінальному кроці генерується список усіх можливих коренів за формулою (x0 + i * n1) % n, що автоматизує перебір кандидатів на ключ.

2. За допомогою програми обчислення частот біграм, яка написана в ході виконання комп'ютерного практикуму №1, знайти 5 найчастіших біграм запропонованого шифртексту (за варіантом).

```
with open("06.txt", "r", encoding="utf-8") as f:
    ciphertext = f.read().replace( _old: '\n',  _new: '').strip()
cipher_bigrams = [ciphertext[i:i + 2] for i in range(0, len(ciphertext), 2)]
counter = collections.Counter(cipher_bigrams)
top_5_cipher = [b for b, c in counter.most_common(5)]
print("Топ-5 біграм шифртексту:", top_5_cipher)
top_5_ru = ["ст", "но", "то", "на", "ен"]
```

Для виконання другого завдання зчитаний із файлу текст розбивається на масив біграм, що не перетинаються, за допомогою генератора списків із кроком 2 (вираз [ciphertext[i:i+2]...]). Для ефективного підрахунку кількості входжень кожної пари літер використано стандартний клас collections.Counter. Його метод most_common(5) автоматично сортує частотний словник за спаданням та витягує п'ять найпопулярніших біграм шифртексту. Отримані значення зберігаються у список top_5_cipher для подальшого формування системи рівнянь та зіставлення з еталонними найчастішими біграмами російської мови («ст», «но», «то», «на», «ен»), які паралельно ініціалізуються у списку top_5_ru.

```
I:\cryptography\.venv\Scripts\python.exe I:\cryptography\.venv\task1.py
Топ-5 біграм шифртексту: ['ще', 'хе', 'чв', 'ле', 'цв']
```

Після запуску скрипта алгоритм успішно обробив вміст зашифрованого файлу та вивів п'ять пар символів, які зустрічаються в ньому найчастіше: «ще», «хе», «чв», «ле» та «цв».

3. Перебрати можливі варіанти співставлення частих біграм мови та частих біграм шифртексту (розглядаючи пари біграм із п’яти найчастіших). Для кожного співставлення знайти можливі кандидати на ключ (a,b) шляхом розв’язання системи (1).

Алгоритм ґрунтується на тому, що афінне біграмне шифрування має вигляд $Y = aX + b \pmod{m^2}$, де $m = 31$ – розмір алфавіту, $m^2 = 961$, а біграма $(x1, x2)$ кодується числом $X = x1 \cdot m + x2$. Ключ (a, b) шукається за відповідністю біграм відкритого тексту (X) та шифртексту (Y).

Для перебору було взято п’ять найчастіших біграм російської мови: «ст», «но», «то», «на», «ен», та п’ять найчастіших біграм шифртексту: «ще», «хе», «чв», «ле», «цв».

Усі біграми переводяться в числа за допомогою функції bigram_to_num, після чого формуються списки X_vals та Y_vals.

```
def bigram_to_num(b):  3 usages
    return alphabet.index(b[0]) * m + alphabet.index(b[1])
```

Для визначення ключа достатньо двох пар (X1, Y1) та (X2, Y2). Віднімаючи друге рівняння системи від першого, отримуємо лінійне порівняння $a \cdot (X1 - X2) \equiv (Y1 - Y2) \pmod{m^2}$. У програмі за допомогою itertools.permutations вибираються всі впорядковані пари різних біграм відкритого тексту ($5 \cdot 4 = 20$

варіантів) та всі впорядковані пари біграм шифртексту (також 20 варіантів), тобто всього перебирається 400 співставлень.

```
possible_keys = set()

for X_pair in itertools.permutations(X_vals, r: 2):
    for Y_pair in itertools.permutations(Y_vals, r: 2):
        X1, X2 = X_pair
        Y1, Y2 = Y_pair

        dX = (X1 - X2) % m2
        dY = (Y1 - Y2) % m2

        a_candidates = solve_linear_congruence(dX, dY, m2)
        for a in a_candidates:
            if ext_gcd(a, m2)[0] == 1:
                b = (Y1 - a * X1) % m2
                possible_keys.add((a, b))

print(f"Кількість кандидатів на ключ: {len(possible_keys)}")
```

Для кожного співставлення обчислюються різниці $dX = (X1 - X2) \bmod m^2$ та $dY = (Y1 - Y2) \bmod m^2$, а порівняння $a \cdot dX \equiv dY \pmod{m^2}$ розв'язується функцією solve_linear_congruence з пункту 1.
Оскільки $m^2 = 31^2$, НСД(dX, 961) може дорівнювати лише 1, 31 або 961. Якщо НСД = 1, порівняння має єдиний розв'язок; якщо НСД = 31 і dY ділиться на 31, розв'язків 31, і функція повертає їх усі; в інших випадках розв'язків немає. Для кожного знайденого кандидата a перевіряється умова взаємної простоти з m^2 (ext_gcd(a, m2)[0] == 1), без якої обернений елемент не існує і розшифрування неможливе. Далі з першого рівняння системи знаходиться $b = (Y1 - a \cdot X1) \bmod m^2$. Отримані пари (a, b) записуються до множини possible_keys, що автоматично усуває дублікати, які виникають при різних співставленнях.
У результаті виконання цього етапу програма виводить повідомлення про кількість кандидатів на ключ: 348.

```
Топ-5 біграм шифртексту: ['ще', 'хе', 'чв', 'ле', 'цв']
Кількість кандидатів на ключ: 348
```

4. Для кожного кандидата на ключ (a,b) розшифрувати шифртекст і за допомогою критерію розпізнавання змістовного тексту відсіяти хибні ключі.

Для кожного кандидата обчислюється обернений елемент a^{-1} за модулем m^2 за допомогою modinv (якщо він не існує, кандидат пропускається). Потім кожна біграма шифртексту Y переводиться в число і розшифровується за

формулою $X = a^{-1} \cdot (Y - b) \bmod m^2$, після чого число перетворюється назад у біграму функцією num_to_bigram.

```
33      def num_to_bigram(n):  1 usage
34          return alphabet[n // m] + alphabet[n % m]
```

Розшифровані біграми конкатенуються у текст-кандидат.
Оскільки кандидатів на ключ багато, перевіряти їх вручну неможливо, тому було реалізовано автоматичний розпізнавач змістовного тексту is_valid_russian, який поєднує два критерії:
1) критерій заборонених біграм: текст відхиляється, якщо в ньому зустрічається хоча б одна біграма зі списку, що неможлива у російській мові («ав», «еь», «иь», «оь», «уь», «ыь», «эь», «юь», «яь», «ьь», «ьы», «йь», «жы», «шы»);
2) критерій частих літер: у змістовному російському тексті літери «о» та «е» зустрічаються часто, тому текст відхиляється, якщо частота «о» нижча за 5 % або частота «е» нижча за 4 %.

```
46      forbidden = ['ав', 'еь', 'иь', 'оь', 'уь', 'ыь', 'эь', 'юь', 'яь', 'ьь', 'ьы', 'йь', 'жы', 'шы']
47
48      def is_valid_russian(text):  1 usage
49          for fb in forbidden:
50              if fb in text: return False
51          o_freq = text.count('о') / len(text)
52          e_freq = text.count('е') / len(text)
53          if o_freq < 0.05 or e_freq < 0.04: return False
54          return True
55
56      Y_vals = [bigram_to_num(b) for b in top_5_cipher]
57      X_vals = [bigram_to_num(b) for b in top_5_ru]
```

Перший критерій дозволяє швидко відкинути більшість хибних розшифрувань, оскільки для випадкового тексту майже напевно знайдеться заборонена біграма. Другий критерій відсіює тексти, які випадково пройшли першу перевірку, але за статистикою літер не схожі на російську мову.

5. Повторити пункти 3–4 для всіх кандидатів, доки не буде знайдено ключ, що дає змістовний текст, та розшифрувати шифртекст.

Програма послідовно перебирає всіх кандидатів із possible_keys. Для першого ключа, розшифрований текст якого пройшов перевірку is_valid_russian, виводяться значення ключа та повний розшифрований текст, після чого цикл завершується оператором break.
Знайдений ключ: a = 441, b = 310.
Початок розшифрованого тексту:
«утробылотихоегородокутанныйтьмоймирнонежилсявпостелипришлолетоиветербыллетнийтеплоедыханиемиранеспешноеиленивоестоитлишьвстатьвысунутьсявокошкоитотчаспоймешьвотонаначинаетсянастоящаясвободаижизньвотонопервоеутролета…»



Це початок російського перекладу роману Р. Бредбері «Вино з кульбаб», тобто ключ знайдено правильно.

1. Які статистичні властивості шифртексту були використані під час формування кандидатів на ключ?

Використано частоти біграм. Афінне біграмне шифрування замінює кожну біграму відкритого тексту завжди однією й тією ж біграмою шифртексту. Тому розподіл частот біграм зберігається, і найчастіші біграми шифртексту («ще», «хе», «чв», «ле», «цв») мають відповідати найчастішим біграмам російської мови («ст», «но», «то», «на», «ен»). Підрахунок зроблено через collections.Counter для непересічних біграм

2. Чому не кожне зіставлення частих біграм відкритого тексту та шифртексту приводить до допустимого ключа?

- Порівняння $a\cdot(X1 - X2) \equiv (Y1 - Y2) \pmod{961}$ може не мати розв'язків. Так буває, коли НСД(dX, 961) не ділить dY.
- Знайдений a може не бути взаємно простим з 961, тобто ділитися на 31. Тоді a^{-1} не існує, розшифрування неможливе, і код відкидає такого кандидата.
- Навіть формально коректний ключ може бути хибним, якщо біграма шифртексту насправді відповідає іншій біграмі відкритого тексту, ніж припущено. Такі ключі відсіює розпізнавач змістовного тексту.

3. Чи виникали під час криптоаналізу випадки, коли одне зіставлення біграм давало декілька кандидатів на ключ? Як такі кандидати оброблялися?

   Такі випадки можливі, і код їх враховує. Модуль $m^2 = 961 = 31^2$ складений, тому НСД(dX, 961) може дорівнювати 1, 31 або 961:

- якщо НСД = 1, порівняння має один корінь;
- якщо НСД = 31 і dY ділиться на 31, коренів 31, і solve_linear_congruence повертає їх усі;
- якщо dY не ділиться на 31, коренів немає.

Кожен корінь a перевірявся на взаємну простоту з 961 (ext_gcd(a, m2)[0] == 1). Для підходящих a обчислювалось $b = (Y1 - a\cdot X1) \bmod 961$, а пара (a, b) додавалась у множину possible_keys. Множина прибирає дублікати, що виникають при різних зіставленнях. Згідно з виводом програми, підсумкова кількість кандидатів дорівнює 348 («Кількість кандидатів на ключ: 348»).

4. За якими ознаками серед отриманих кандидатів було визначено правильний ключ?

Це єдиний ключ із 348, розшифрований текст якого пройшов автоматичну перевірку. Він дає (a, b) = (441, 310). Підтверджує його і сам текст: це зв'язний російський текст без заборонених біграм.

5. Який критерій автоматичного розпізнавання осмисленого тексту було використано? Чому було обрано саме його?

Використано два критерії:

- **Заборонені біграми:** текст відхиляється, якщо містить хоча б одну з біграм «аь», «еь», «иь», «оь», «уь», «ыь», «эь», «юь», «яь», «ьь», «ъы», «йь», «жы», «шы».
- **Частоти літер:** «о» має бути не менше 5 %, а «е» не менше 4 %.

Критерій заборонених біграм обрано, бо він простий, швидко відсікає випадковий текст і не потребує еталонної статистики. Критерій частот «о» і «е» додано як страховку.

6. Як поводився обраний критерій для правильного відкритого тексту та для результатів розшифрування неправильними ключами?

Серед перевірок використовувались заборонені біграми та частота появи літер «о» та «е». Правильний ключ зміг пройти обидві перевірки, тоді як неправильні ключі відсіювались на одному з двух критеріїв.

Висновки:

У ході виконання комп’ютерного практикуму було здійснено криптоаналіз афінного біграмного шифру за варіантом 6. Реалізовано підпрограми розширеного алгоритму Евкліда, пошуку оберненого елемента за модулем та розв’язування лінійних порівнянь із поверненням усіх розв’язків, що є необхідним, коли НСД(a, m^2) = 31. Знайдено п’ять найчастіших біграм шифртексту («ще», «хе», «чв», «ле», «цв») та зіставлено їх із найчастішими біграмами російської мови («ст», «но», «то», «на», «ен»).
Перебір 400 співставлень пар біграм дав множину кандидатів на ключ, яку було перевірено автоматично за допомогою розпізнавача, що поєднує критерій заборонених біграм і критерій частот літер «о» та «е». Це дозволило визначити ключ (a, b) = (441, 310) і отримати змістовний розшифрований текст. Слід зазначити, що застосований розпізнавач є досить простим: програма зупиняється на першому ключі, що пройшов перевірку, тому в принципі можливе хибне спрацювання.